Двоичная куча

Конспект к тесту по АиСД: определения, операции, buildHeap за O(n) с доказательством, HeapSort, типовые задачи и вопросы для самопроверки.

0 Что повторить: чек-лист

- ☐ Определение кучи, max-heap и min-heap — §1
- ☐ Индексы детей и родителя (от 0 и от 1) — §1
- ☐ Высота кучи = $\lfloor \log_2 n \rfloor$, доказательство — §1
- ☐ Листья: индексы $\lfloor n/2 \rfloor \ldots n-1$ — §1
- ☐ siftDown: условие применения, код, корректность — §2
- ☐ siftUp: код, корректность — §2
- ☐ insert, extractMax, delete(i), change key — §3
- ☐ Таблица сложностей всех операций — §3
- ☐ **buildHeap: алгоритм и пример вручную** — §4
- ☐ **buildHeap: инвариант и корректность** — §4
- ☐ **buildHeap за O(n): доказательство** — §4
- ☐ Почему n вставок дают $\Theta(n \log n)$ — §4
- ☐ HeapSort: код, $\Theta(n \log n)$, O(1) памяти, неустойчивость — §5
- ☐ d-ичная куча, std::priority_queue — §6
- ☐ Типовые задачи: top-k, слияние k массивов, медиана — §7
- ☐ Вопросы для самопроверки — §8

> **Если времени мало:** §1 → §2 → §4 целиком → таблица в конце §3 → §8.

1 Определение и хранение в массиве

> **Определение. Двоичная куча** (пирамида, heap) — это **почти полное** (завершённое) двоичное дерево: все уровни заполнены полностью, кроме, может быть, последнего, а последний заполнен **слева направо** без пропусков. Дополнительно выполняется **основное свойство кучи**:
>
> - **Max-heap:** значение в каждой вершине $\geqslant$ значений в её детях. Следовательно, в корне лежит максимум, а в каждом поддереве его корень — максимум этого поддерева.
> - **Min-heap:** значение в каждой вершине $\leqslant$ значений детей, в корне минимум.

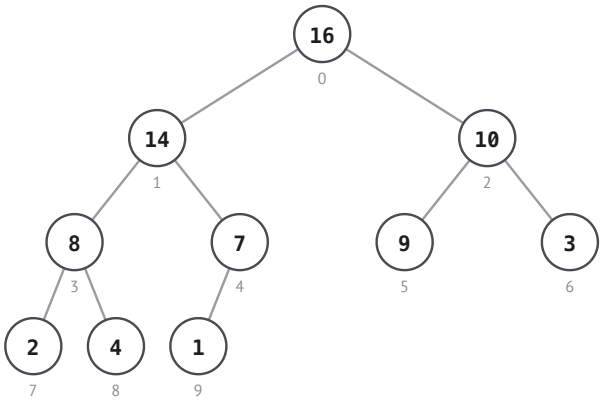
Дальше везде **max-heap**. Для min-heap все сравнения меняются на противоположные.

Важно: куча *не* упорядочивает элементы между братьями и между разными ветками. Это не дерево поиска: левый ребёнок может быть как больше, так и меньше правого.

Хранение в массиве

Из-за формы «без дыр» дерево хранится прямо в массиве по уровням, слева направо. Указатели не нужны.

	Нумерация с 0 (как в лекции)	Нумерация с 1 (Кормен)
Корень	`A[0]`	`A[1]`
Левый ребёнок `i`	`2i + 1`	`2i`
Правый ребёнок `i`	`2i + 2`	`2i + 1`
Родитель `i`	`(i − 1) / 2` (вниз)	`i / 2` (вниз)
Внутренние вершины	`0 … ⌊n/2⌋ − 1`	`1 … ⌊n/2⌋`
Листья	`⌊n/2⌋ … n − 1`	`⌊n/2⌋ + 1 … n`



Пример max-кучи из 10 элементов

16	14	10	8	7	9	3	2	4	1

Под каждым узлом подписан его индекс в массиве.

Высота кучи

Утверждение. Высота кучи из n элементов равна $H = \lfloor \log_2 n \rfloor$.

Уровни 0…H−1 заполнены полностью, это $1 + 2 + \ldots + 2^{H-1} = 2^H - 1$ вершин. На уровне H есть от 1 до 2^H вершин. Значит,

$$2^H \le n \le 2^{H+1} - 1 < 2^{H+1} \quad \Longrightarrow \quad H \le \log_2 n < H + 1,$$

то есть $H = \lfloor \log_2 n \rfloor$. Отсюда: любая операция, которая проходит один путь корень—лист, работает за O(log n).

■

Где листья

Утверждение. При нумерации с 0 вершина `i` имеет ребёнка ⇔ `2i + 1 ≤ n − 1` ⇔ `i ≤ ⌊n/2⌋ − 1`. Значит, листья — это `⌊n/2⌋ … n − 1`, их ровно `⌈n/2⌉` (примерно половина).

Следствия:

- в max-куче **минимум лежит в одном из листьев**: у внутренней вершины есть ребёнок, а он не больше её. Найти минимум можно только перебором листьев, за Θ(n);
- **второй максимум** лежит в `A[1]` или `A[2]`;
- buildHeap начинает с `⌊n/2⌋ − 1`: это последняя вершина, у которой есть ребёнок.

2 Базовые операции: siftDown и siftUp

Все остальные операции собираются из двух «просеиваний».

siftDown (просеивание вниз, heapify)

Когда применять. Свойство кучи нарушено **только в вершине i**, а **оба поддерева её детей уже являются кучами**. Иначе siftDown не обязан починить кучу.

Идея: сравниваем `A[i]` с **бóльшим** из детей `A[j]`. Если `A[i] ≥ A[j]`, всё в порядке, останавливаемся. Иначе меняем их местами и продолжаем из `j`.

```
// Max-куча хранится в A[0..heapSize-1]
void siftDown(vector<int>& A, int i, int heapSize) {
    while (2 * i + 1 < heapSize) {            // пока есть хотя бы левый ребёнок
        int j = 2 * i + 1;                    // левый ребёнок
        if (j + 1 < heapSize && A[j + 1] > A[j])
            j++;                              // j — больший из детей
        if (A[i] >= A[j]) break;              // свойство выполнено — стоп
        swap(A[i], A[j]);
        i = j;                                // продолжаем с позиции ребёнка
    }
}
```

> **Почему именно с бóльшим ребёнком?** После обмена новый родитель должен быть ⩾ обоих детей. Если поднять меньшего ребёнка, он окажется над бóльшим братом, и свойство сразу нарушится.

Корректность (индукция по высоте *i*). Пусть `A[j]` — больший ребёнок и `A[i] < A[j]`. После обмена:

- в вершине `i` стоит максимум из трёх чисел, поэтому он ⩾ обоих детей. Кроме того, `A[j]` был максимумом своего поддерева, значит, он ⩾ всего, что ниже;
- поддерево второго ребёнка не изменилось и осталось кучей;
- в поддереве `j` свойство может нарушиться только в самой вершине `j`, а её поддеревья — кучи. Это та же ситуация, но на меньшей высоте. По предположению индукции siftDown(j) её починит.

Сложность: O(h), где h — высота вершины i. Каждый шаг — это 2 сравнения и 1 обмен, то есть O(1). В худшем случае O(log n). Итеративная версия использует O(1) памяти, рекурсивная — O(log n) на стек.

siftUp (просеивание вверх)

> **Когда применять.** Свойство кучи нарушено только между `A[i]` и его родителем: `A[i]` слишком большой. Например, после добавления элемента в конец или после увеличения ключа.

```
void siftUp(vector<int>& A, int i) {
    while (i > 0 && A[(i - 1) / 2] < A[i]) {   // родитель меньше — меняем
        swap(A[i], A[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}
```

Корректность: пусть `p` — родитель `i` и `A[p] < A[i]`. До нарушения `A[p]` был ⩾ всех в своём поддереве, кроме, может быть, самого `A[i]`. После обмена `A[i]` оказывается над старым `A[p]` и над братом. Он больше обоих, так что внизу всё в порядке. Нарушение может остаться только между новой позицией и её родителем, и мы продолжаем подниматься.

Сложность: O(глубина i) ⩽ O(log n). На шаг приходится 1 сравнение.

3 Операции очереди с приоритетом

```
int getMax(const vector<int>& A) { return A[0]; }                  // O(1)

void insert(vector<int>& A, int key) {                              // O(log n)
    A.push_back(key);                    // кладём в первую свободную позицию
    siftUp(A, A.size() - 1);             // и поднимаем
}

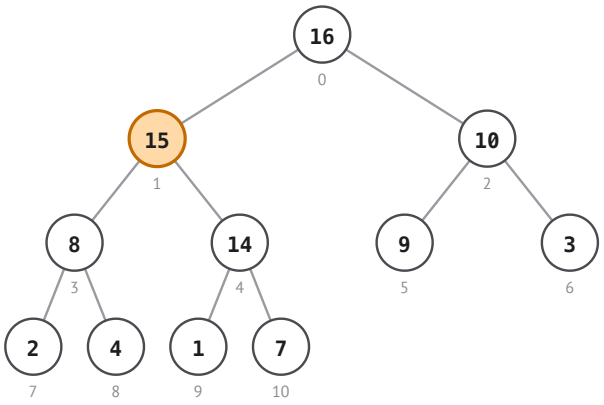
int extractMax(vector<int>& A) {                                    // O(log n)
    int mx = A[0];
    A[0] = A.back();                     // последний элемент — в корень
    A.pop_back();
    if (!A.empty()) siftDown(A, 0, A.size());
    return mx;
}

void erase(vector<int>& A, int i) {      // delete(i); delete — ключевое слово C++
    A[i] = A.back();
    A.pop_back();
    if (i < (int)A.size()) {             // если удаляли не последний
        siftUp(A, i);                    // сработает не больше одного
        siftDown(A, i, A.size());        // из двух просеиваний
    }
}

void increaseKey(vector<int>& A, int i, int key) {   // key ≥ A[i]
    A[i] = key;
    siftUp(A, i);
}
void decreaseKey(vector<int>& A, int i, int key) {   // key ≤ A[i]
    A[i] = key;
    siftDown(A, i, A.size());
}
```

Пример: insert(15)

Кладём 15 в конец (индекс 10). Её родитель (индекс 4) равен 7 < 15, меняем. Новый родитель (индекс 1) равен 14 < 15, меняем. Следующий родитель — корень 16 > 15, стоп.

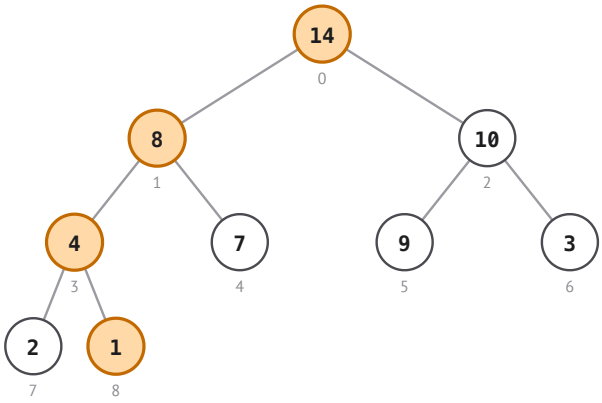


insert(15): путь индексов 10 → 4 → 1

16	15	10	8	14	9	3	2	4	1	7

Пример: extractMax()

Из кучи `[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]` забираем 16. Последний элемент (1) ставим в корень и просеиваем вниз, каждый раз выбирая бóльшего ребёнка.

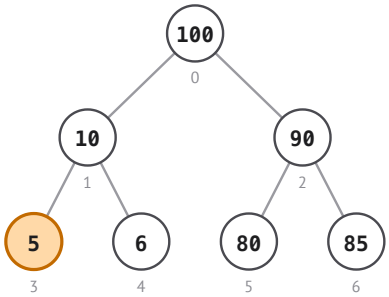


extractMax = 16: элемент 1 прошёл 0 → 1 → 3 → 8

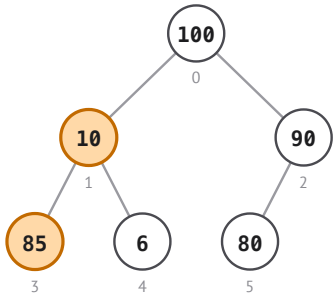
14	8	10	4	7	9	3	2	1

Пример: delete(i), где нужен именно siftUp

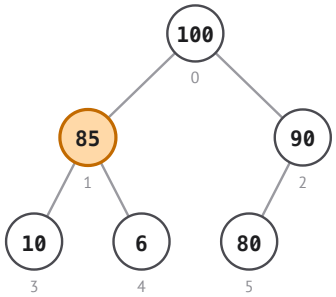
Ловушка. После удаления произвольного элемента последний элемент, поставленный на его место, может оказаться **больше родителя**, потому что он пришёл из другой ветки. Поэтому одного siftDown недостаточно. Нужно проверить оба направления. Для корня (extractMax) достаточно siftDown, потому что родителя у корня нет.



Удаляем A[3] = 5



85 на месте 3 больше родителя 10!



После siftUp(3)

Сводная таблица сложностей

Операция	Как делается	Время
getMax	`A[0]`	O(1)
insert	в конец + siftUp	O(log n)
extractMax	последний → корень, siftDown	O(log n)
delete(i)	последний → i, siftUp *или* siftDown	O(log n)
increaseKey / decreaseKey	siftUp / siftDown	O(log n)
buildHeap	siftDown для i = ⌊n/2⌋−1 … 0	**Θ(n)**
Поиск произвольного элемента	перебор	Θ(n)
Минимум в max-куче	перебор листьев	Θ(n)
Слияние куч размеров n и m	склеить массивы + buildHeap	Θ(n + m)
HeapSort	buildHeap + (n−1)·extractMax	Θ(n log n), O(1) памяти

Для delete(i) и changeKey нужно знать *индекс* элемента. Если известно только значение, сначала придётся искать его за Θ(n) или хранить отдельный массив позиций.

4 Построение кучи за O(n): buildHeap

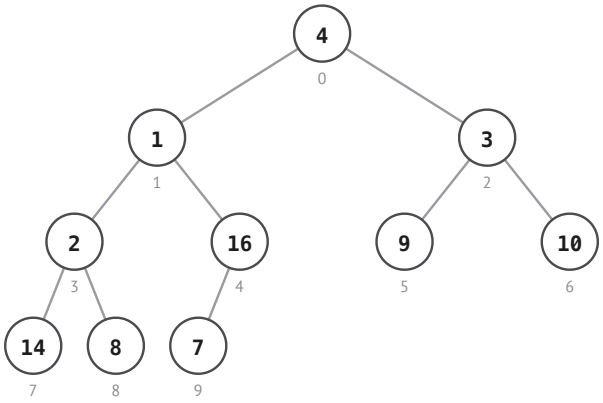
Алгоритм

Дан произвольный массив. Вызываем siftDown для всех вершин, у которых есть дети, **снизу вверх**, то есть от последней внутренней к корню.

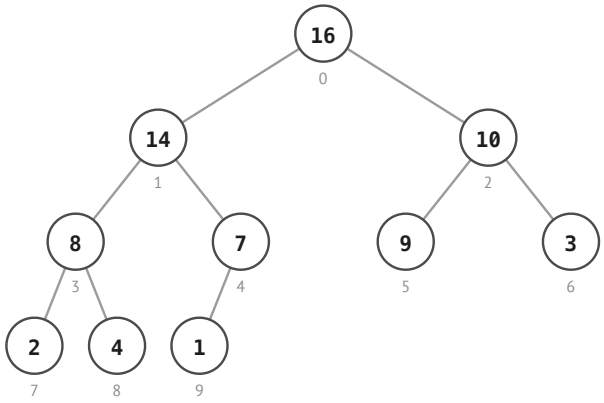
```
void buildHeap(vector<int>& A) {
    int n = A.size();
    for (int i = n / 2 - 1; i >= 0; i--)       // листья (i ≥ n/2) пропускаем: они уже кучи
        siftDown(A, i, n);
}
```

Пример вручную

Массив `[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]`, n = 10, начинаем с i = 10/2 − 1 = 4. В таблице показано состояние массива **после** каждого вызова, затронутые позиции подсвечены.



Исходный массив



После buildHeap

Вызов	Обмены (индексы)	Шагов	Массив после
siftDown(4)	—	0	4 1 3 2 **16** 9 10 14 8 7
siftDown(3)	3↔7	1	4 1 3 **14** 16 9 10 **2** 8 7
siftDown(2)	2↔6	1	4 1 **10** 14 16 9 **3** 2 8 7
siftDown(1)	1↔4, 4↔9	2	4 **16** 10 14 **7** 9 3 2 8 **1**
siftDown(0)	0↔1, 1↔3, 3↔8	3	**16** **14** 10 **8** 7 9 3 2 **4** 1

Всего обменов: **7** при n = 10. Обратите внимание: siftDown(0) опускает 4 через три уровня, а вызовы для нижних вершин делают по 0–1 шагу.

Корректность

Инвариант цикла. Перед итерацией с индексом *i* каждая вершина `i+1, i+2, …, n−1` является корнем корректной кучи.

Инициализация. Перед первой итерацией i = ⌊n/2⌋ − 1, и вершины ⌊n/2⌋…n−1 — листья. Лист — тривиальная куча.
Сохранение. Дети вершины i имеют индексы 2i+1 и 2i+2, оба больше i. По инварианту их поддеревья — кучи. Это ровно условие применимости siftDown, поэтому после siftDown(i) поддерево i — тоже куча. Поддеревья вершин i+1, … не затронуты, кроме тех, что лежат внутри поддерева i, а оно теперь целиком куча. После i-- инвариант снова выполнен.
Завершение. После итерации i = 0 корень (вершина 0) — корень кучи, то есть весь массив — куча. ∎

Ловушка: порядок обхода. Если вызывать siftDown для i = 0, 1, 2, … (сверху вниз), результат может **не быть кучей**. В момент вызова для корня поддеревья его детей ещё не кучи, и условие siftDown не выполнено. Пример: `[1, 2, 3, 4]`. siftDown(0) даёт `[3, 2, 1, 4]`, siftDown(1) даёт `[3, 4, 1, 2]`, а 4 > 3. Это не куча.

Почему наивная оценка O(n log n) грубая

Вызовов siftDown около n/2, каждый за O(log n), то есть O(n log n). Это *верная*, но *не точная* верхняя оценка. Каждый siftDown работает за **высоту своей вершины**, а не за log n. Большинство вершин находится внизу, и их высота маленькая: половина — листья (высота 0, не вызываются), четверть — высота 1, восьмая часть — высота 2 и т. д.

Доказательство O(n) — способ 1: по уровням

Теорема. buildHeap выполняет меньше 2n шагов siftDown (обменов), то есть работает за O(n). Так как каждый элемент нужно хотя бы прочитать, время Θ(n).

Пусть H = ⌊$\log_2 n$⌋ — высота кучи, и, как показано в §1, $\mathbf{2^H \leqslant n}$.

- На глубине *k* не больше 2^k вершин.
- siftDown из вершины глубины *k* делает не больше *H* − *k* шагов: каждый шаг опускает элемент на уровень, а ниже уровня H опускаться некуда.

Суммарное число шагов:

$$T \le \sum_{k=0}^{H} 2^k (H-k) \overset{j=H-k}{=} \sum_{j=0}^{H} j \cdot 2^{H-j} = 2^H \sum_{j=0}^{H} \frac{j}{2^j} < 2 \cdot 2^H \le 2n\,.$$

Осталось обосновать, что сумма $\sum \frac{j}{2^j}$ меньше 2 (лемма ниже). Каждый шаг siftDown — это 2 сравнения и 1 обмен, то есть O(1). Значит, всё время O(n). ∎

Лемма.

$$\sum_{j=0}^{H} \frac{j}{2^j} = 2 - \frac{H+2}{2^H} < 2, \qquad \sum_{j=0}^{\infty} \frac{j}{2^j} = 2\,.$$

Способ «вычесть половину» (для бесконечного ряда; он сходится, например, по признаку Даламбера):

$$S = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \ \dots\,, \quad \frac{S}{2} = \frac{1}{4} + \frac{2}{8} + \ \dots \quad \Longrightarrow \quad S - \frac{S}{2} = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \ \dots \ = 1 \quad \Longrightarrow \quad S = 2\,.$$

Конечная формула — индукцией по H. База H = 0: слева 0, справа 2 − 2/1 = 0. Переход: прибавим к обеим частям (H+1)/2^{H+1}:

$$2 - \frac{H+2}{2^H} + \frac{H+1}{2^{H+1}} = 2 - \frac{2H+4-H-1}{2^{H+1}} = 2 - \frac{(H+1)+2}{2^{H+1}}\,.$$

∎

Доказательство O(n) — способ 2: по высотам (как в Кормене)

Лемма. В куче из n элементов не больше $\lceil n / 2^{h+1} \rceil$ вершин высоты h.

Индукция по h. **h = 0:** вершины высоты 0 — это листья, их $\lceil n/2 \rceil$ (§1).
Переход h−1 → h. Удалим из кучи все листья. Останутся вершины $0 \dots \lfloor n/2 \rfloor - 1$, то есть снова дерево «формы кучи» из $n' = \lfloor n/2 \rfloor$ вершин. Высота каждой оставшейся вершины уменьшилась ровно на 1: каждый путь вниз заканчивался листом, и он потерял последнюю вершину. Значит, вершины высоты h в старом дереве — это вершины высоты h−1 в новом. По предположению индукции их не больше

$$\left\lceil \frac{n'}{2^h} \right\rceil = \left\lceil \frac{\lfloor n/2 \rfloor}{2^h} \right\rceil \le \left\lceil \frac{n}{2^{h+1}} \right\rceil .$$

Оценка. siftDown из вершины высоты h стоит не больше c·h. Для $h \leqslant H$ выполнено $n/2^{h+1} \geqslant n/2^{H+1} \geqslant 1/2$, а для $x \geqslant 1/2$ верно $\lceil x \rceil \leqslant 2x$. Поэтому ∎

$$T \le \sum_{h=0}^{H} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot ch \le cn \sum_{h=0}^{H} \frac{h}{2^h} < 2cn = O(n)\,.$$

Точное значение для полного дерева

Если дерево полное, то есть $n = 2^{H+1} - 1$, сумма высот всех вершин равна ровно **$n - (H+1) = n - \log_2(n+1)$**. Значит, buildHeap делает **меньше n обменов** и **меньше 2n сравнений**. Проверка: при H = 1 (n = 3) сумма 1 = 3 − 2, при H = 2 (n = 7) сумма 2 + 1 + 1 = 4 = 7 − 3.

Почему построение через n вставок даёт Θ(n log n)

Альтернатива: пустая куча и n раз `insert`, то есть siftUp. Это **O(n log n)**, и на плохом входе оценка достигается.

siftDown стоит **высоту** вершины. Большинство вершин внизу, у них высота мала, поэтому сумма даёт O(n).

siftUp стоит **глубину** вершины. Большинство вершин внизу, у них глубина ≈ log n, поэтому сумма даёт Θ(n log n).

Худший вход: возрастающий массив 1, 2, …, n в max-кучу. Каждый новый элемент — новый максимум, и он всплывает до корня, то есть делает $\lfloor \log_2 i \rfloor$ обменов, где i — его номер (с 1). У последних n/2 элементов $i > n/2$, поэтому $\lfloor \log_2 i \rfloor \geqslant \lfloor \log_2 n \rfloor - 1$. Только они уже дают $\geqslant (n/2) \cdot (\lfloor \log_2 n \rfloor - 1) = \Omega(n \log n)$ обменов.

n (возрастающий массив)	обменов в buildHeap	обменов при n×insert
15	11	34
1 023	1 013	8 194
1 000 000	999 988	17 951 445

5 HeapSort (пирамидальная сортировка)

Идея: это сортировка выбором, в которой поиск максимума за O(n) заменён извлечением из кучи за O(log n). Чтобы сортировать по **возрастанию**, берём **max**-кучу: максимум уходит в конец массива, на своё окончательное место.

```
void heapSort(vector<int>& A) {
    int n = A.size();
    buildHeap(A);                                  // Фаза 1: O(n)
    for (int heapSize = n; heapSize > 1; heapSize--) {
        swap(A[0], A[heapSize - 1]);               // максимум — в конец
        siftDown(A, 0, heapSize - 1);              // чиним кучу меньшего размера
    }                                              // Фаза 2: (n-1)·O(log n)
}
```

Инвариант фазы 2: `A[0..heapSize−1]` — max-куча, `A[heapSize..n−1]` содержит **n − heapSize наибольших** элементов в отсортированном порядке, и каждый из них ⩾ любого элемента кучи.

Пример

Сортируем `[5, 2, 8, 1, 9, 3]`. Серым показана уже отсортированная часть, она не входит в кучу.

Этап	Массив
buildHeap	9 5 8 1 2 3
шаг 1: max = 9 → A[5]	8 5 3 1 2 9
шаг 2: max = 8 → A[4]	5 2 3 1 8 9
шаг 3: max = 5 → A[3]	3 2 1 5 8 9
шаг 4: max = 3 → A[2]	2 1 3 5 8 9
шаг 5: max = 2 → A[1]	1 2 3 5 8 9

Свойства

- **Время:** O(n log n) всегда: O(n) на построение плюс n−1 извлечение по O(log n). На различных элементах Θ(n log n) даже в лучшем случае. В отличие от сортировки вставками, «почти отсортированный» вход не ускоряет HeapSort.
- **Память:** O(1) дополнительной, сортировка на месте (in-place) при итеративном siftDown.
- **Неустойчива.** Пример: $[2_a, 2_b, 1]$ — это уже куча. После сортировки получаем $[1, 2_b, 2_a]$, равные двойки поменялись местами.
- Это сортировка сравнениями, поэтому быстрее Ω(n log n) сравнений она быть не может.

- На практике медленнее quicksort из-за плохой локальности: прыжки `i → 2i+1` бьют по кэшу. Зато у неё гарантированный худший случай, поэтому IntroSort переключается на HeapSort, когда рекурсия quicksort слишком глубока.

6 Вариации

d-ичная куча

У каждой вершины d детей. Нумерация с 0: дети `d·i + 1 … d·i + d`, родитель `(i − 1) / d`. Высота ≈ $\log_d n$.

Операция	Время	Почему
siftUp, insert, increaseKey	$O(\log_d n)$	одно сравнение на уровень
siftDown, extractMax	$O(d \cdot \log_d n)$	на уровне ищем максимум среди d детей
buildHeap	O(n)	при константном d

Применение: если вставок и decreaseKey намного больше, чем извлечений (например, Дейкстра на плотном графе), бóльшее d выгоднее.

std::priority_queue

```
priority_queue<int> pq;                                    // max-куча (по умолчанию!)
priority_queue<int, vector<int>, greater<int>> minpq;      // min-куча
pq.push(x);   pq.top();   pq.pop();   pq.size();           // O(log n), O(1), O(log n), O(1)
// Конструктор от диапазона строит кучу за O(n):  priority_queue<int> pq(a.begin(), a.end());
// Также есть std::make_heap / push_heap / pop_heap / sort_heap в <algorithm>.
```

7 Типовые задачи

1. Проверить, является ли массив max-кучей — O(n)

```
bool isMaxHeap(const vector<int>& A) {
    for (int i = 1; i < (int)A.size(); i++)
        if (A[(i - 1) / 2] < A[i]) return false;   // каждый ребёнок ≤ родителя
    return true;
}
```

Полезный факт: массив, отсортированный **по убыванию**, всегда max-куча, а **по возрастанию** — min-куча. Обратное неверно.

2. k наибольших элементов из n

- **Способ A:** buildHeap (max) и k раз extractMax. Время **O(n + k log n)**.
- **Способ B:** держать **min**-кучу размера k. Для каждого x: push, и если размер > k, делаем pop (выкидываем наименьший). В конце в куче k наибольших, а её вершина — k-й по величине. Время **O(n log k)**, память O(k), работает на потоке данных.

```cpp
int kthLargest(const vector<int>& a, int k) {
    priority_queue<int, vector<int>, greater<int>> pq;    // min-куча
    for (int x : a) {
        pq.push(x);
        if ((int)pq.size() > k) pq.pop();
    }
    return pq.top();
}
```

3. Слияние k отсортированных массивов (суммарно N элементов) — O(N log k)

Кладём в min-кучу по первому элементу каждого массива, храним пары `(значение, номер массива)`. Извлекаем минимум, пишем его в ответ и кладём следующий элемент из того же массива. В куче всегда не больше k элементов.

4. Медиана потока чисел

Две кучи: **max**-куча `L` для меньшей половины и **min**-куча `R` для большей. Поддерживаем `|L| = |R|` или `|L| = |R| + 1` и условие `L.top() ≤ R.top()`. Медиана — это `L.top()` или среднее двух вершин. Добавление работает за O(log n), запрос медианы — за O(1).

5. Ручная трассировка

Самая частая задача на тестах: выполнить buildHeap, insert, extractMax или один шаг HeapSort и выписать массив. Алгоритм: нарисовать дерево по уровням, подписать индексы и двигаться строго по правилам из §2–§4. Внимание: при siftDown меняем с **бóльшим** ребёнком, а при равенстве детей берём левого (по коду выше).

8 Вопросы для самопроверки

1. Сколько элементов может быть в куче высоты h?
→ От 2^h до $2^{h+1} - 1$.

2. Сколько листьев в куче из n элементов и какие у них индексы (с 0)?
→ $\lceil n/2 \rceil$ листьев, индексы $\lfloor n/2 \rfloor$ … n−1.

3. Где может находиться минимальный элемент max-кучи? А второй по величине? Третий?
→ Минимум — в любом листе. Второй — в A[1] или A[2]. Третий — на глубине 1 или 2 (индексы 1…6), но не в корне.

4. Сформулируйте условие, при котором siftDown(i) гарантированно даёт кучу.
→ Поддеревья обоих детей i — кучи, нарушение возможно только в самой вершине i.

5. Почему в buildHeap цикл идёт от $\lfloor n/2 \rfloor - 1$ вниз до 0, а не от 0 вверх?
→ Чтобы к моменту вызова siftDown(i) поддеревья детей уже были кучами (инвариант). Сверху вниз это не так, контрпример — [1, 2, 3, 4].

6. Почему buildHeap — O(n), хотя вызывается ~n/2 раз siftDown за O(log n)?
→ siftDown стоит высоту вершины, а не log n. Сумма высот $\Sigma\, 2^k(H-k) < 2n$ (§4).

7. Сколько стоит построение кучи n вставками? Приведите худший вход.
→ Θ(n log n). Возрастающий массив для max-кучи: каждый элемент всплывает до корня.

8. Почему при удалении произвольного элемента может понадобиться siftUp?
→ На его место ставится последний элемент из другой ветки, и он может быть больше нового родителя (пример в §3).

9. Почему при siftDown меняем с бóльшим ребёнком?
→ Иначе меньший ребёнок окажется родителем бóльшего, и свойство нарушится.

10. Сколько сравнений на один уровень делают siftDown и siftUp?

→ siftDown — 2 (дети между собой и родитель с бóльшим), siftUp — 1. Отсюда асимметрия в d-ичной куче.

11. Устойчив ли HeapSort? Сколько дополнительной памяти он использует?

→ Неустойчив. O(1) дополнительной памяти.

12. Какую кучу использовать в HeapSort для сортировки по возрастанию?

→ Max-кучу: максимум переставляется в конец, на своё место.

13. Являются ли max-кучами массивы [10, 8, 9, 3, 7, 2, 1], [10, 7, 9, 3, 8, 2, 1] и [10, 7, 9, 6, 3, 2, 11]?

→ Первый — да. Второй — нет: A[4] = 8 > A[1] = 7. Третий — нет: A[6] = 11 > A[2] = 9.

14. Можно ли найти произвольный элемент в куче быстрее, чем за O(n)?

→ Нет: куча не упорядочивает ветки между собой, в худшем случае смотреть придётся всё.

15. Как слить две кучи размеров n и m?

→ Склеить массивы и вызвать buildHeap: Θ(n + m). Вставки по одной дали бы O(m log(n + m)).

16. Как через кучу найти k-й по величине элемент за O(n log k)?

→ Min-куча размера k, при переполнении выбрасываем минимум. Ответ — вершина (§7).